

Отчёт по лабораторной работе №14

Архитектура компьютеров и операционные системы

Степан Андреевич Гусев

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	8
4	Выполнение лабораторной работы	10
4.1	Первый скрипт	10
4.2	Второй скрипт	11
4.3	Третий скрипт	13
5	Выводы	15
6	Ответы на контрольные вопросы	16
7	Список литературы	19

Список иллюстраций

4.1	Создание файла 1	10
4.2	Скрипт 1	11
4.3	Содержимое каталога	12
4.4	Создание файла 2	12
4.5	Скрипт 2	12
4.6	Справка sat	13
4.7	Создание файла 3	13
4.8	Скрипт 3	13
4.9	Результат работы скрипта 3	14

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

- 1) Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой (`> /dev/tty#`, где `#` — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов.
- 2) Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`.
- 3) Используя встроенную переменную `$RANDOM`, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. Учтите, что `$RANDOM` выдаёт псевдослучайные числа в диапа-

зоне от 0 до 32767.

3 Теоретическое введение

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек:

оболочка Борна (Bourne shell или sh) — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций;

C-оболочка (или csh) — надстройка на оболочкой Борна, использующая C-подобный синтаксис команд с возможностью сохранения истории выполнения команд;

оболочка Корна (или ksh) — напоминает оболочку C, но операторы управления программой совместимы с операторами оболочки Борна;

BASH — сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек C и Корна (разработка компании Free Software Foundation).

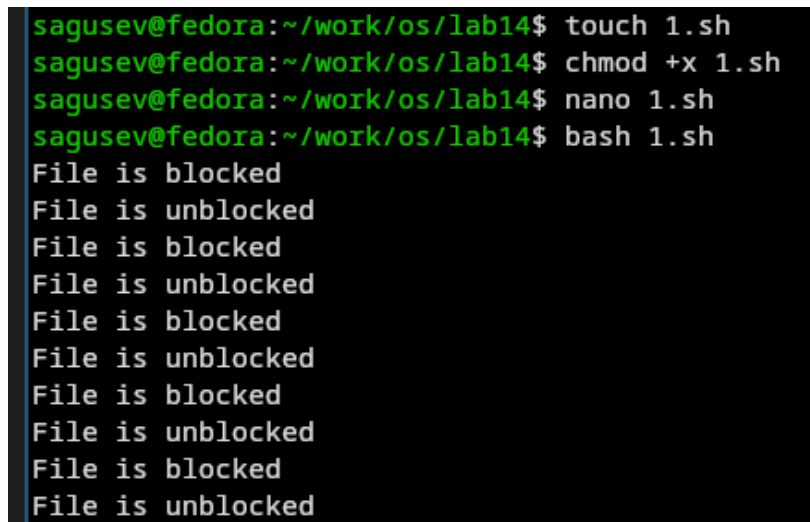
POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ. Стандарты POSIX разработаны комитетом IEEE (Institute of Electrical and Electronics Engineers) для обеспечения совместимости различных UNIX/Linux-подобных операционных систем и переносимости прикладных программ на уровне исходного кода. POSIX-совместимые оболочки разработаны на базе оболочки Корна. Рассмотрим основные элементы программирования в оболочке bash. В других оболочках большинство команд

будет совпадать с описанными ниже.

4 Выполнение лабораторной работы

4.1 Первый скрипт

Создал файл 1.sh, дал права на выполнение, ввёл код и запустил скрипт (рис. 4.1).



```
sagusev@fedora:~/work/os/lab14$ touch 1.sh
sagusev@fedora:~/work/os/lab14$ chmod +x 1.sh
sagusev@fedora:~/work/os/lab14$ nano 1.sh
sagusev@fedora:~/work/os/lab14$ bash 1.sh
File is blocked
File is unblocked
File is blocked
File is unblocked
File is blocked
File is unblocked
File is blocked
File is unblocked
File is blocked
File is unblocked
File is blocked
File is unblocked
```

Рисунок 4.1: Создание файла 1

Текст скрипта 1.sh (рис. 4.2).

```
nano 1.sh
GNU nano 8.3
#!/bin/bash
lockfile="./lock.file"
exec {fn}>$lockfile
timesleep=3

while test -f "$lockfile"
do
if flock -n ${fn}
then
    echo "File is blocked"
    sleep $timesleep
    echo "File is unblocked"
    flock -u ${fn}
else
    echo "File is unlocked"
    sleep $timesleep
fi
done
```

Рисунок 4.2: Скрипт 1

4.2 Второй скрипт

Изучил содержимое каталога /usr/share/man/man1 (рис. 4.3).

```
~/work/os/lab14
sagusev@fedora:~/work/os/lab14$ ls /usr/share/man/man1
.:1.gz          mktexmf.1.gz
'[:1.gz'        mktexpk.1.gz
7z.1.gz         mktexfm.1.gz
a2ping.1.gz     mlabel.1.gz
abrt.1.gz       mm2gv.1.gz
abrt-action-analyze-backtrace.1.gz  mmafm.1.gz
abrt-action-analyze-c.1.gz          mmcli.1.gz
abrt-action-analyze-ccpp-local.1.gz  mmd.1.gz
abrt-action-analyze-oops.1.gz        mmdblookup.1.gz
abrt-action-analyze-python.1.gz      mmount.1.gz
abrt-action-analyze-vmcore.1.gz       mmove.1.gz
abrt-action-analyze-vulnerability.1.gz mmpfb.1.gz
abrt-action-analyze-xorg.1.gz         modulemd-validator.1.gz
abrt-action-check-oops-for-hw-error.1.gz modutil.1.gz
abrt-action-find-bodhi-update.1.gz    moggsplit.1.gz
```

Рисунок 4.3: Содержимое каталога

Создал файл 2.sh, дал права на выполнение (рис. 4.4).

```
sagusev@fedora:~/work/os/lab14$ touch 2.sh
sagusev@fedora:~/work/os/lab14$ chmod +x 2.sh
sagusev@fedora:~/work/os/lab14$ nano 2.sh
sagusev@fedora:~/work/os/lab14$ bash 2.sh cat
```

Рисунок 4.4: Создание файла 2

Текст скрипта 2.sh (рис. 4.5).

```
nano 2.sh
GNU nano 8.3
#!/bin/bash

a=$1

if test -f "/usr/share/man/man1/$a.1.gz"
then
    less /usr/share/man/man1/$a.1.gz
else
    echo "No such command"
fi
```

Рисунок 4.5: Скрипт 2

После запуска открылось окно со справкой для команды cat (рис. 4.6).

```
bash 2.sh cat
ESC[4mCATESC[24m(1)                                User Commands
ESC[4mCATESC[24m(1)

ESC[1mNAMEESC[0m
cat - concatenate files and print on the standard output

ESC[1mSYNOPSISESC[0m
ESC[1mcat ESC[22m[ESC[4mOPTIONESC[24m]... [ESC[4mFILEESC[24m]...

ESC[1mDESCRIPTIONESC[0m
Concatenate FILE(s) to standard output.

With no FILE, or when FILE is -, read standard input.

ESC[1m-AESC[22m, ESC[1m--show-allESC[0m
equivalent to ESC[1m-vETESC[0m

ESC[1m-bESC[22m, ESC[1m--number-nonblankESC[0m
number nonempty output lines, overrides ESC[1m-nESC[0m
```

Рисунок 4.6: Справка cat

4.3 Третий скрипт

Создал файл 3.sh, дал права на выполнение, открыл файл (рис. 4.7).

```
sagusev@fedora:~/work/os/lab14$ touch 3.sh
sagusev@fedora:~/work/os/lab14$ chmod +x 3.sh
sagusev@fedora:~/work/os/lab14$ nano 3.sh
```

Рисунок 4.7: Создание файла 3

Ввёл текст скрипта 3.sh (рис. 4.8).

```
nano 3.sh
GNU nano 8.3 3.sh
#!/bin/bash

a=$1

for ((i=0; i<$a; i++))
do
    ((char=$RANDOM%26+1))
    case $char in
        1) echo -n a;; 2) echo -n b;; 3) echo -n c;; 4) echo -n d;; 5) echo -n e;; 6) echo -n f;;
        7) echo -n g;; 8) echo -n h;; 9) echo -n i;; 10) echo -n j;; 11) echo -n k;; 12) echo -n l;;
        13) echo -n m;; 14) echo -n n;; 15) echo -n o;; 16) echo -n p;; 17) echo -n r;; 18) echo -n s;;
        19) echo -n t;; 20) echo -n q;; 21) echo -n u;; 22) echo -n v;; 23) echo -n w;; 24) echo -n x;;
        25) echo -n y;; 26) echo -n z;;
    esac
done
echo
```

Рисунок 4.8: Скрипт 3

Запустил скрипт 3 и проверил, что всё сработало корректно (рис. 4.9).

```
sagusev@fedora:~/work/os/lab14$ bash 3.sh 3  
eom  
sagusev@fedora:~/work/os/lab14$ bash 3.sh 3  
rdz  
sagusev@fedora:~/work/os/lab14$ bash 3.sh 0  
  
sagusev@fedora:~/work/os/lab14$ bash 3.sh 30  
mxrvrmimwgsjtzoeocuhvliyscalkk  
sagusev@fedora:~/work/os/lab14$ |
```

Рисунок 4.9: Результат работы скрипта 3

5 Выводы

Я изучил основы программирования в оболочке ОС UNIX и научился писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

6 Ответы на контрольные вопросы

- 1) В данной строчке допущены следующие ошибки: не хватает пробелов после первой скобки [и перед второй скобкой] выражение \$1 необходимо взять в “ ”, потому что эта переменная может содержать пробелы Таким образом, правильный вариант должен выглядеть так: while [«\$1» != «exit»]
- 2) Чтобы объединить несколько строк в одну, можно воспользоваться несколькими способами: Первый: VAR1=«Hello,» VAR2=» World” VAR3=«VAR1VAR2» echo «VAR3” : *Hello,World* : VAR1 = *”Hello,”VAR1+ = ”World”echo”VAR1»* Результат: Hello, World
- 3) Команда seq в Linux используется для генерации чисел от ПЕРВОГО до ПОСЛЕДНЕГО шага INCREMENT. Параметры: seq LAST: если задан только один аргумент, он создает числа от 1 до LAST с шагом шага, равным 1. Если LAST меньше 1, значение is не выдает. seq FIRST LAST: когда заданы два аргумента, он генерирует числа от FIRST до LAST с шагом 1, равным 1. Если LAST меньше FIRST, он не выдает никаких выходных данных. seq FIRST INCREMENT LAST: когда заданы три аргумента, он генерирует числа от FIRST до LAST на шаге INCREMENT. Если LAST меньше, чем FIRST, он не производит вывод. seq -f «FORMAT» FIRST INCREMENT LAST: эта команда используется для генерации последовательности в форматированном виде. FIRST и INCREMENT являются необязательными. seq -s «STRING» ПЕРВЫЙ ВКЛЮЧЕНО: Эта команда используется для STRING

для деления чисел. По умолчанию это значение равно /n. FIRST и INCREMENT являются необязательными. seq -w FIRST INCREMENT LAST: эта команда используется для выравнивания ширины путем заполнения начальными нулями. FIRST и INCREMENT являются необязательными.

- 4) Результатом данного выражения $\$(10/3)$ будет 3, потому что это целочисленное деление без остатка.
- 5) Отличия командной оболочки zsh от bash: В zsh более быстрое автодополнение для cd с помощью Tab, В zsh существует калькулятор zcalc, способный выполнять вычисления внутри терминала, В zsh поддерживаются числа с плавающей запятой, В zsh поддерживаются структуры данных «хэш», В zsh поддерживается раскрытие полного пути на основе неполных данных, В zsh поддерживается замена части пути, В zsh есть возможность отображать разделенный экран, такой же как разделенный экран vim
- 6) for ((a=1; a <= LIMIT; a++)) синтаксис данной конструкции верен, потому что, используя двойные круглые скобки, можно не писать \$ перед переменными ().
- 7) Преимущества и недостатки скриптового языка bash:
 - Один из самых распространенных и ставится по умолчанию в большинстве дистрибутивах Linux, MacOS
 - Удобное перенаправление ввода/вывода
 - Большое количество команд для работы с файловыми системами Linux
 - Можно писать собственные скрипты, упрощающие работу в LinuxНедостатки скриптового языка bash:
 - Дополнительные библиотеки других языков позволяют выполнить больше действий

- Bash не является языков общего назначения
- Утилиты, при выполнении скрипта, запускают свои процессы, которые, в свою очередь, отражаются на скорости выполнения этого скрипта
- Скрипты, написанные на bash, нельзя запустить на других операционных системах без дополнительных действий

7 Список литературы

1. https://esystem.rudn.ru/pluginfile.php/3097185/mod_resource/content/4/012-lab_shell_prog_3.pdf